

Parser and Runtime Environment Design Report

1. Agreement on the Intermediate Form

The parser and the runtime environment both agree on the Abstract Syntax Tree (AST) as the intermediate form that bridges the parsing and execution phases of the Matrix Language interpreter.

- Parser Phase:

Implemented in `parser.y` and supported by `lexer.l`, the parser is designed to construct an AST rather than directly executing code. Grammar rule actions instantiate AST node classes like `NumberNode`, `IfNode`, `ForNode`, `WhileNode`, and others, all defined in `ast.h`. The result of a successful parse is a pointer to the root of this tree, referred to as `astRoot`.

- Runtime Phase:

Implemented through the `Interpreter` class (`interpreter.h/.cpp`) and the virtual `execute()` methods in each AST node (`ast.cpp`), the runtime environment traverses the AST to execute the program. Execution begins at `astRoot`, and node-specific logic handles constructs like loops, conditionals, and matrix operations by evaluating or executing subtrees recursively.

This shared use of the AST forms a clean, modular interface between parsing and interpretation, enabling clarity, testability, and extensibility in both stages.

2. Correctness of Runtime Execution

Once the AST is successfully built and the program compiles, the runtime should correctly interpret programs written in the Matrix Language — including arithmetic operations, logical conditions, and control flow constructs like `if`, `while`, and `for`.

- Execution Logic:

Each AST node's `execute()` method contains logic corresponding to its construct (e.g., `ForNode::execute()` handles iteration). These methods utilize supporting structures like the symbol table and `Value` class to manage runtime state and data types.

- Verification Strategy:

Execution correctness must be confirmed through systematic testing. A suite of sample input files — such as `test_arithmetic.txt`, `test_if_else.txt`, `test_while.txt`, etc. — should be executed

using the compiled matrixlang binary. Output from each test should be compared to expected results to validate runtime behavior.

Example Command:

```
amish130437@Amishs-MacBook-Air SER502-MatLan-Team36 % ./src/matrixlang ./data/test1.matlan
```

Starting parse

Entering state 0

Stack now 0

Reading a token

Next token is token IDENTIFIER ()

Shifting token IDENTIFIER ()

Entering state 1

Stack now 0 1

Reading a token

Next token is token ASSIGN ()

Shifting token ASSIGN ()

Entering state 16

Stack now 0 1 16

Reading a token

Next token is token NUMBER ()

Shifting token NUMBER ()

Entering state 20

Stack now 0 1 16 20

Reducing stack by rule 46 (line 213):

\$1 = token NUMBER ()

-> \$\$ = nterm primaryExpr ()

P.S: Full output is not given as it is too long.

3. Parse Tree Debugging and AST Visualization

Parse Tree (Concrete Syntax Tree)

The parser does not explicitly build a concrete parse tree. However, Bison offers internal debugging features to trace parsing behavior:

- Run the parser, and Bison will print detailed parsing steps (e.g., token shifts, rule reductions) to standard error, providing insight into how the grammar is applied.

4. Separation of Concerns

The project clearly separates the parsing logic from the runtime execution, in line with compiler design best practices.

Component	Responsibility	Key Files
Parser	Builds AST from source program	<code>parser.y</code> , <code>lexer.l</code> , <code>ast.h</code>
AST Classes	Represent syntactic structure and execute nodes	<code>ast.h</code> , <code>ast.cpp</code>
Interpreter	Traverses AST and performs program execution	<code>interpreter.h</code> , <code>interpreter.cpp</code>
Utilities	Matrix operations and helper functions	<code>matrix.cpp</code> , <code>utils.cpp</code>

Conclusion

The system implements a robust, maintainable interpreter architecture using a two-phase approach: AST construction followed by AST execution. The parser and runtime share a clearly defined interface via the AST, enabling clean separation of concerns. With comprehensive test coverage and debugging support, the Matrix Language interpreter is well-positioned for correctness, extensibility, and clarity.